

Introduction

In **concurrent programming**, managing threads can quickly become complicated. While threads provide a way to execute multiple tasks simultaneously, creating and managing threads manually is often inefficient and error-prone in real-world systems. Let's understand this with the help of an example.

Understanding

Imagine building a **ride-matching** system like **Uber**, where the system needs to handle requests from drivers and riders simultaneously. Each time a rider requests a ride, the system has to find an available driver. To do this efficiently, the system may need to handle multiple tasks **concurrently**, such as processing requests, matching riders with drivers, updating their statuses, and sending notifications.

In a simple approach, you might decide to create a new thread every time a ride request comes in, so each request gets processed independently. The code implementing this approach is given below:

```
import java.util.*;

// Ride Matching Service Class

class RideMatchingService {

    // Method handling ride request

    public void requestRide(String riderId) {
```

```

// Creating a new thread for the ride

Thread matchThread = new Thread(() -> {

    System.out.println("Matching rider " + riderId + " to a driver...");

    // Simulate some processing

    try {

        Thread.sleep(1000); // Simulate a 1-second matching process

    } catch (InterruptedException e) {

        Thread.currentThread().interrupt();

    }

    System.out.println("Ride matched for rider " + riderId);

});

matchThread.start();

}

}

```

```

class Main {

    public static void main(String[] args) {

        RideMatchingService rideService1 = new RideMatchingService();

        RideMatchingService rideService2 = new RideMatchingService();
    }
}

```

```
rideService1.requestRide("Raj");

System.out.println("task1 running...");


rideService2.requestRide("John Doe");

System.out.println("task2 running...");

}

}
```

While this seems like a good idea at first, creating and managing a large number of threads can quickly become a nightmare as the system scales. Let's formally understand why we should not create thread manually in real-world systems.

Issues while Creating Thread Manually in Real-World Systems

- **Thread Explosion:** Creating a new thread for each task can quickly lead to an excessive number of threads, overwhelming the system and degrading performance.
- **Memory Issue:** Each thread consumes system memory for its stack. Creating too many threads can lead to memory exhaustion, causing crashes or slowdowns.
- **Thread Leaks:** Failing to properly terminate threads results in thread leaks, where unused threads continue consuming resources, leading to performance degradation.
- **Context Switching Overhead:** Managing too many threads increases context switching, where the system saves and loads thread states. This overhead reduces overall system efficiency as the CPU spends more time switching between threads than doing real computational work.

A Better Approach: Thread Pools

A better approach is to use **Thread Pools**, where a **fixed number of threads are available to handle tasks**.

Real-life Analogy

This is similar to hiring a set number of chefs for a kitchen: instead of hiring a new chef every time an order comes in, you use your existing chefs efficiently to handle multiple orders as they come in. This ensures that resources are optimized and the kitchen (or system) runs smoothly without overloading.

Executor Framework

The **Executor Framework** in Java is a **high-level API** that provides a simple and flexible mechanism for managing and controlling thread execution. It decouples:

- **Task Submission:** What you want to do.
- **Task Execution:** How and when it runs.

allowing you to manage threads more efficiently. This framework is part of Java's `java.util.concurrent` package and simplifies the process of handling threads. Let's us understand the key concepts of **Executor Frameworks** first.

1. Executor Interface

The **Executor interface** is the foundation of the **Executor Framework**. It defines a single method:

- `void execute(Runnable command)`: The **execute** method takes a **Runnable** task and runs it **asynchronously** in a thread. It doesn't

return any result, and you can't track the execution outcome directly.

2. **ExecutorService Interface**

ExecutorService extends the **Executor** interface and adds methods for managing and controlling the lifecycle of threads.

Key Methods:

- **submit(Callable task)**: Returns a **Future** object which allows you to track the progress of the task.
- **shutdown()**: Initiates an orderly shutdown of the executor, rejecting any new tasks but processing the already submitted ones.
- **shutdownNow()**: Tries to stop all actively executing tasks and halts the processing of waiting tasks.

3. **ThreadPoolExecutor Class**

ThreadPoolExecutor is a concrete implementation of **ExecutorService** and is one of the most commonly used classes. It allows you to define a pool of threads to execute tasks.

Key Parameters:

- **corePoolSize**: The number of threads to keep in the pool.
- **maximumPoolSize**: The maximum number of threads allowed in the pool.
- **keepAliveTime**: Time for which idle threads will be kept alive before being terminated.
- **workQueue**: A queue used to hold tasks before they are executed.

4. **Executors Class**

Executors is a utility class that provides factory methods to create predefined types of executor services.

Common Methods:

- **newFixedThreadPool(int nThreads)**: Creates a thread pool with a fixed number of threads.
- **newCachedThreadPool()**: Creates a thread pool that can dynamically grow and shrink.
- **newSingleThreadExecutor()**: Creates a single-threaded executor.

Code

Here's a code snippet using the **Executor Framework** to handle **email sending tasks** efficiently.

```
import java.util.concurrent.*;

// Using the newFixedThreadPool to manage threads

class EmailService {

    private static final ExecutorService executor =
Executors.newFixedThreadPool(10); // Thread pool with 10 threads

    // Method to send email

    public static void sendEmail(String recipient) {

        executor.execute(() -> {

            System.out.println("Sending email to " + recipient + " on " +
Thread.currentThread().getName());
```

```

    try {
        // Simulate dummy work (sending an email)
        Thread.sleep(1000); // Simulate delay
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt(); // Handle interruption
    }

    System.out.println("Email sent to " + recipient);
});
}

// Main method to simulate sending multiple emails
public static void main(String[] args) {
    for (int i = 1; i <= 25; i++) {
        sendEmail("user" + i + "@gmail.com"); // Send email to 1000 users
    }

    executor.shutdown(); // Gracefully shut down the executor
}
}

```

Understanding

- **ExecutorService executor = Executors.newFixedThreadPool(10);**

This creates a thread pool with 10 threads. It means the system can process a maximum of 10 email sending tasks concurrently.

- `executor.execute(() -> { ... });`

This method is used to submit a task to the executor. The task here is a `Runnable` that sends an email. The task is executed by one of the threads from the thread pool.

- `Thread.sleep(1000);`

This simulates some dummy work (like sending an email). The thread sleeps for 1 second to simulate the time it takes to send an email.

- `executor.shutdown();`

This initiates an orderly shutdown of the executor. It ensures that all submitted tasks are completed before the program terminates.

Why Use Executor Frameworks?

- **Simplified Thread Management:** The framework handles thread creation, task execution, and management for you, reducing boilerplate code.
- **Efficient Resource Management:** By using thread pools, you avoid the overhead of creating new threads for every task, leading to better performance.
- **Graceful Shutdown:** Executor services allow for a clean shutdown, ensuring that all tasks complete before the system terminates.

Methods to Submit Tasks

In **Java's Executor Framework**, submitting tasks is a fundamental operation that determines how tasks are executed asynchronously. The `ExecutorService` provides several methods to submit tasks for execution. Let's understand them one by one.

1: `execute()` Method

- **Purpose:** It is used to submit **Runnable** tasks (tasks that do not return any result).
- **Return Type:** It does not return a result. This method simply submits the task for execution and doesn't provide a way to track the result or exceptions.
- **Usage:** It's ideal for tasks where you don't need a result back from the task (e.g., logging, sending an email, etc.).

Note that we've studied this method in the previous section.

2: `submit()` Method

- **Purpose:** It is used to submit both **Runnable** and **Callable** tasks (tasks that can return a result).
- **Return Type:** It returns a **Future** object, which allows you to track the result and handle exceptions thrown during the task's execution.
- **Usage:** It's ideal for tasks where you need to capture and process the result (e.g., performing calculations or retrieving data).

Let's understand the `submit()` method through a detailed example in the next section.

When to Use Each:

- `execute()` is best used for **fire-and-forget** tasks where you don't need a result back and don't need to track task completion.
- `submit()` is used when you need to **retrieve a result** or **track progress** with the help of a Future object.

`submit()` method

In this section, we'll break down how the `submit()` method works and how it interacts with the `Future` object to manage task results. Let's understand this better through the below code:

```
import java.util.concurrent.*;

// Future and Submit example

class FutureExample {

    public static void main(String[] args) throws Exception {

        ExecutorService executor = Executors.newFixedThreadPool(2);

        Future<Integer> future = executor.submit(() -> {

            Thread.sleep(1000);

            return 77;

        });

        System.out.println("Doing other work...");

        Integer result = future.get(); // blocks until result is ready

        System.out.println("Result: " + result);

        executor.shutdown();

    }
}
```

```
}
```

Key Concepts to Understand

1. `submit()` Method:

- The `submit()` method is a versatile way to submit tasks to an executor. It accepts both **Runnable** and **Callable** tasks:
 - **Runnable**: A task that doesn't return a result.
 - **Callable**: A task that can return a result (like our example where 77 is returned).
- The method returns a **Future** object, which can be used to monitor the task's progress and retrieve the result once it completes.

2. **Future** Object:

The **Future** object represents the result of an asynchronous computation. It allows you to:

- **Track the completion** of the task.
- **Get the result** using `get()`, which blocks the calling thread until the task finishes.
- **Cancel the task** if it hasn't started yet.

3. `get()` Method:

- The `get()` method of the **Future** object is **blocking**, meaning it will pause the calling thread until the task has finished executing and the result is ready.
- If the task has already completed, `get()` will return the result immediately. Otherwise, it will wait for the task to finish.

4. Graceful Shutdown:

The `shutdown()` method is used to stop the executor service after all submitted tasks have been completed. It's important to always call `shutdown()` to release resources and properly shut down the thread pool.

Why Use `submit()` and `Future`?

- **Asynchronous Task Execution:** `submit()` allows tasks to run asynchronously, meaning they won't block the main thread. This is useful for performing background tasks such as handling user requests, processing data, etc.
 - **Result Handling:** The `Future` object provides a way to handle the result of the task, including retrieving the outcome once it's ready.
 - **Control and Flexibility:** You can monitor the progress of the task, cancel it if necessary, and manage its lifecycle effectively.
-

Shutdown Methods in Executor Class

The `ExecutorService` in Java provides two important methods for shutting down an executor: `shutdown()` and `shutdownNow()`. These methods are essential for gracefully terminating the execution of tasks and releasing resources when they are no longer needed.

`shutdown()` method

```
executor.shutdown();
```

- **Purpose:** Initiates an orderly shutdown of the executor service. Once this method is called, the executor will stop accepting new tasks but will continue to process the tasks that have already been submitted.
- **Return Type:** `void` (No return value)
- **Usage:**
 - `shutdown()` is typically used when you want to allow the executor to finish executing all tasks in the queue before shutting down.
 - It ensures that the executor doesn't accept any new tasks but still processes the tasks that have already been submitted.

`shutdownNow()` method

```
List<Runnable> pendingTasks = executor.shutdownNow();
```

- **Purpose:** Attempts to stop all actively executing tasks, halts the processing of waiting tasks, and returns a list of the tasks that were waiting to be executed.
 - **Return Type:** `List<Runnable>` — Returns a list of the tasks that were in the executor's queue but were not executed.
 - **Usage:**
 - `shutdownNow()` tries to immediately stop all tasks, including those that are currently running.
 - It does not guarantee that all tasks will be stopped; it may just attempt to interrupt them.
 - It returns a list of tasks that have not yet started executing, so you can handle those tasks or retry them later if needed.
-

Thread Starvation and Fairness

In multi-threaded systems, **thread starvation** and **fairness** are key concepts that ensure threads are scheduled properly without some being unfairly blocked or neglected.

Thread Starvation

Thread starvation occurs when a thread is perpetually unable to access the resources it needs for execution due to high contention, often caused by prioritizing certain threads over others. As a result, low-priority threads or threads with resource dependencies may never get a chance to execute.

Cause of Thread Starvation:

- **High Thread Priority:** Higher-priority threads monopolize CPU time, blocking lower-priority threads.
- **Infinite Blocking:** Threads waiting for resources may never gain access due to constant contention.
- **Lack of Fair Scheduling:** Systems that don't implement fairness may neglect certain threads, causing starvation.

Fairness in Thread Scheduling

Fairness ensures that all threads get an opportunity to execute, preventing some threads from being permanently blocked. A fair scheduler allocates CPU time evenly across all threads, preventing starvation.

Fixes for Thread Safety

- Adjusting Thread Priorities
 - Implementing Fair Locks
 - Using Time Slicing or Round Robin Scheduling
 - Adopting Fair Scheduling Algorithms
 - Leveraging Executor Services for Task Management
-

Fixed vs Cached vs Scheduled Thread Pools

Java's `ExecutorService` provides different types of thread pools that help manage and execute tasks in a concurrent environment. The three most common types of thread pools are **Fixed Thread Pool**, **Cached Thread Pool**, and **Scheduled Thread Pool**. Each type has specific use cases, advantages, and disadvantages.

1. Fixed Thread Pool

A **Fixed Thread Pool** creates a pool with a fixed number of threads. Once a task is submitted, the executor assigns it to an available thread from the pool. If all threads are busy, new tasks are queued until a thread becomes available.

use Case:

Applications where the number of tasks is known in advance, and the system should process a fixed number of concurrent tasks (e.g., handling a fixed number of user requests simultaneously).

Advantages:

- **Predictable resource usage:** The number of threads is fixed, so system resources are predictable.
- **Better control:** You can control the maximum number of concurrent threads, avoiding system overload.

Disadvantages:

- **Limited scalability:** If all threads are busy and the task queue fills up, tasks must wait in the queue, potentially delaying execution.
- **Underutilization:** If fewer tasks are available than the number of threads, some threads may remain idle, wasting system resources.

Cached Thread Pool

A **Cached Thread Pool** creates new threads as needed but reuses previously constructed threads when they are available. If a thread remains idle for more than 60 seconds, it is terminated and removed from the pool.

Use Case:

Short-lived tasks that are executed intermittently, such as handling burst traffic or processing small background tasks where thread usage is unpredictable.

Advantages:

- **Scalable:** Threads are created dynamically, and the pool can grow as needed to handle bursts of tasks.

- **Efficient resource use:** Threads are reused whenever possible, reducing the cost of thread creation.

Disadvantages:

- **Potential for thread explosion:** If the system experiences a high volume of tasks at once, a large number of threads may be created, leading to resource exhaustion.
- **Less predictable resource usage:** The number of threads can fluctuate significantly, making resource management more difficult.

Scheduled Thread Pools

A **Scheduled Thread Pool** allows you to schedule tasks with fixed-rate or fixed-delay execution policies. It supports delayed or periodic execution of tasks, making it useful for scheduling tasks at regular intervals or after a specific delay. Let's understand with the code given:

```
class SessionCleaner {  
  
    public static void main(String[] args) {  
  
        ScheduledExecutorService scheduler =  
Executors.newScheduledThreadPool(1);  
  
        Runnable task = () -> System.out.println("Cleaning up expired  
sessions...");  
  
        scheduler.scheduleAtFixedRate(task, 0, 5, TimeUnit.SECONDS);  
  
    }  
  
}
```

Explanation:

In the above example, a session-cleaning task is scheduled to run **every 5 seconds**, starting immediately (**initialDelay = 0**). This is achieved using **`scheduleAtFixedRate()`**, which ensures periodic execution using a **`ScheduledExecutorService`**.

Use Case:

Applications that need to perform tasks at fixed intervals or after a certain delay, such as periodic data synchronization, background maintenance tasks, or scheduled jobs.

Advantages:

- **Task scheduling:** Allows scheduling tasks with a delay or periodic execution.
- **Flexible:** Provides methods for scheduling tasks at fixed-rate intervals or with a fixed delay between executions.

Disadvantages:

- **Less efficient for short-lived tasks:** Not ideal for tasks that are short-lived or don't need to be scheduled periodically.
- **Thread management overhead:** Managing scheduled tasks requires additional overhead for tracking execution times and intervals.